



ChipWhisperer:

Power traces, leakage,
and side-channel attacks

By: Rupesh Hemant Bhusare, 2203106

Guided by: Prof. Sharad Sinha

Indian Institute of Technology, Goa

AGENDA

- **Problem Statement**
- **Existing Solutions**
- **Proposed Approach**
- **Experiments & Results**
- **Future Work**

PROBLEM STATEMENT

Cryptographic devices often leak information through their **power consumption**. This leakage varies based on:

- the data being processed,
- the instructions executed,
- and internal hardware transitions.

Attackers can measure these variations to infer:

- program flow,
- intermediate values,
- and even **secret keys**.

Problem:

To study how such leakage occurs in real embedded systems and determine whether firmware implementations (like AES) are vulnerable to power analysis attacks.

WHY THIS MATTERS

- AES is widely used in embedded and IoT devices
- Even correct implementations may leak physically
- Power analysis attacks require no code modification
- Understanding leakage helps build secure systems
- Demonstrates the practical risks of side-channel attacks

EXISTING SOLUTIONS & PRIOR WORK

1. Manual Oscilloscope Measurements

- Capture power traces manually
- Hard to synchronize with firmware
- Difficult for beginners

2. Simulation-Based Leakage Models

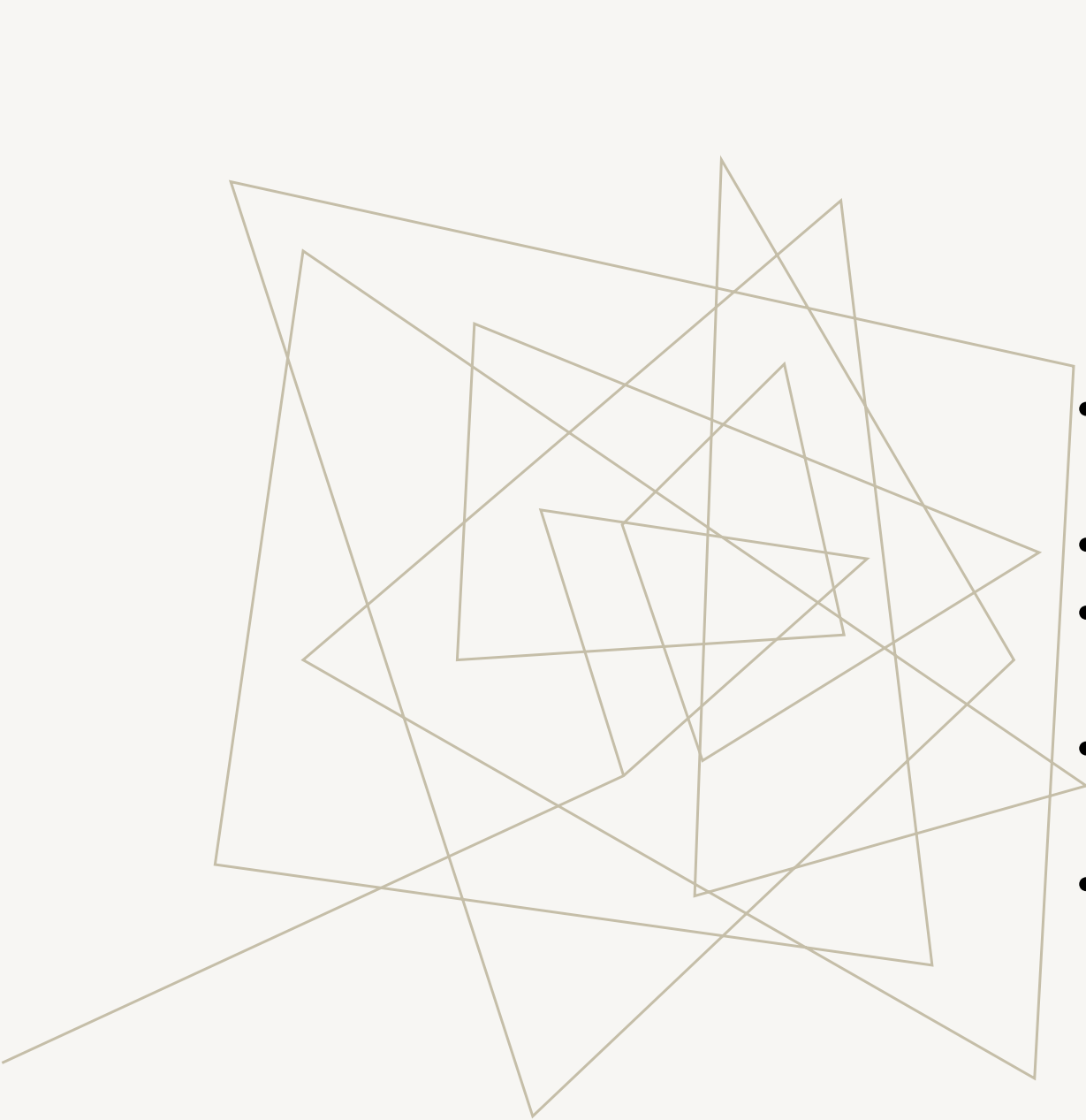
- Predict leakage using Hamming Weight / Hamming Distance
- Helps understand theory but not real hardware behavior

3. Academic Demonstrations

- Small examples showing how leakage occurs
- Usually limited to simulated traces or basic experiments

4. Limited Hardware Experiments

- Some prior labs exist but are scattered
- No complete end-to-end workflow from setup
→ capture → attack

An abstract graphic on the left side of the slide, consisting of several overlapping, thin-lined triangles in a light beige color. The triangles are of various sizes and orientations, creating a complex, layered geometric pattern.

MOTIVATION

- Leakage is hard to understand purely from theory
- Real measurements reveal patterns clearly
- Educational hardware makes SCA accessible
- Goal: build intuition + demonstrate real attacks
- Complete workflow: setup → capture → analyze → break

PROPOSED SOLUTION

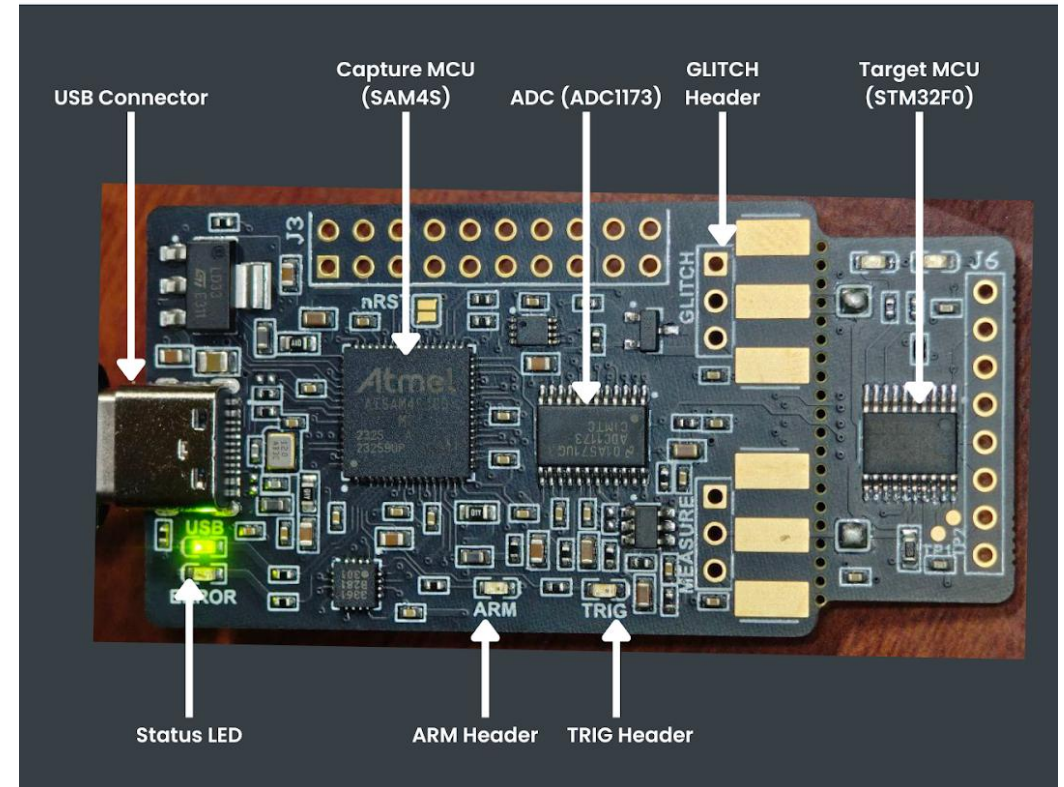
Use **ChipWhisperer Nano** to perform:

- Instruction-level power analysis
- Password bypass using branch leakage
- Hamming Weight experiments
- Differential Power Analysis on firmware AES
- Power vs Hamming Weight characterization

This gives a full end-to-end view of leakage and key extraction.

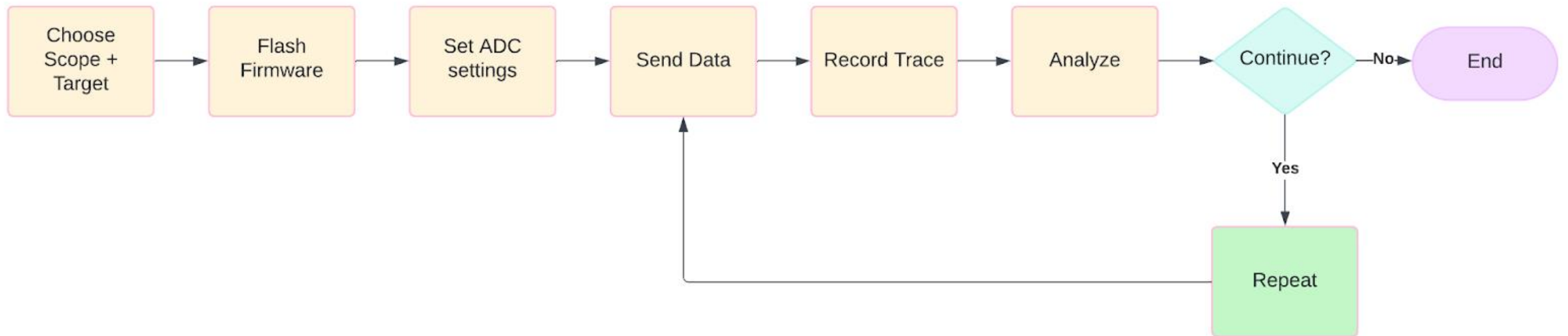
TECHNOLOGY STACK

- ChipWhisperer Nano (scope + target)
- Python + Jupyter Notebooks
- ChipWhisperer API (scope, target, simpleserial)
- TinyAES firmware
- Matplotlib & Bokeh for trace visualization



ChipWhisperer Nano with labelled components

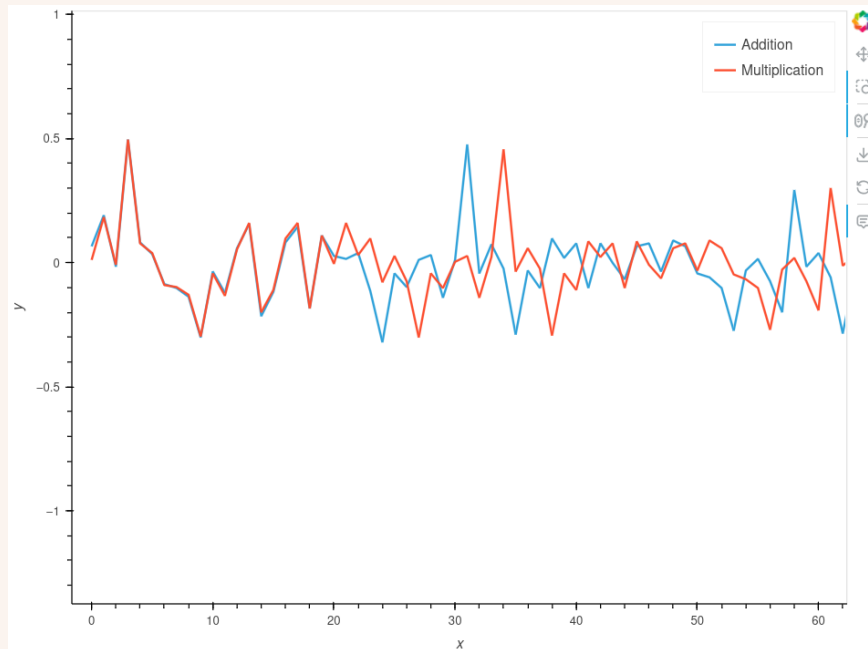
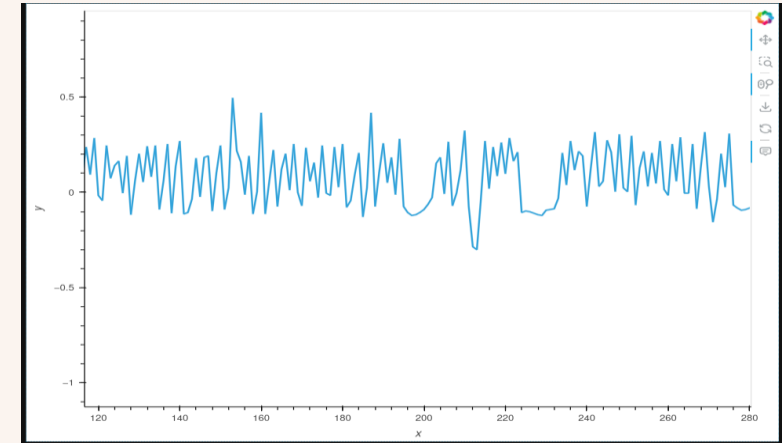
EVERY EXPERIMENT FLOW



Every experiment repeats the same cycle with different inputs.

INSTRUCTION-LEVEL POWER ANALYSIS

we analyze how different CPU instructions affect the power consumption of the microcontroller. The goal is to understand the *baseline leakage*: how the device behaves even before doing AES or cryptography.



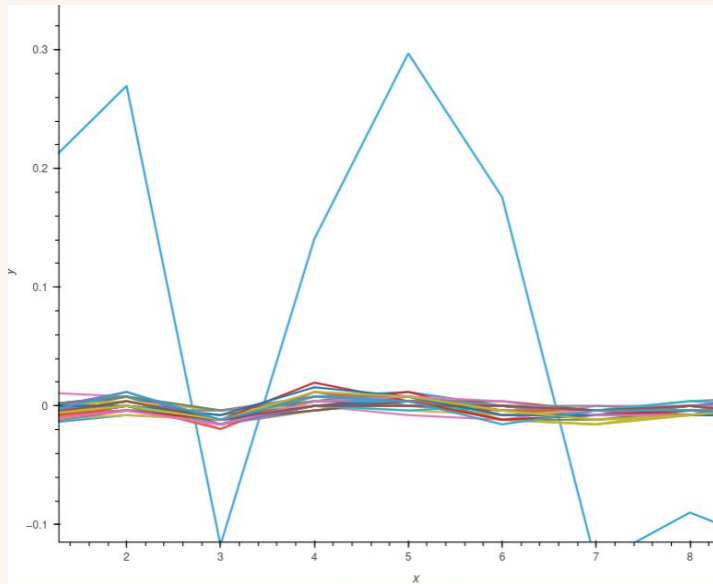
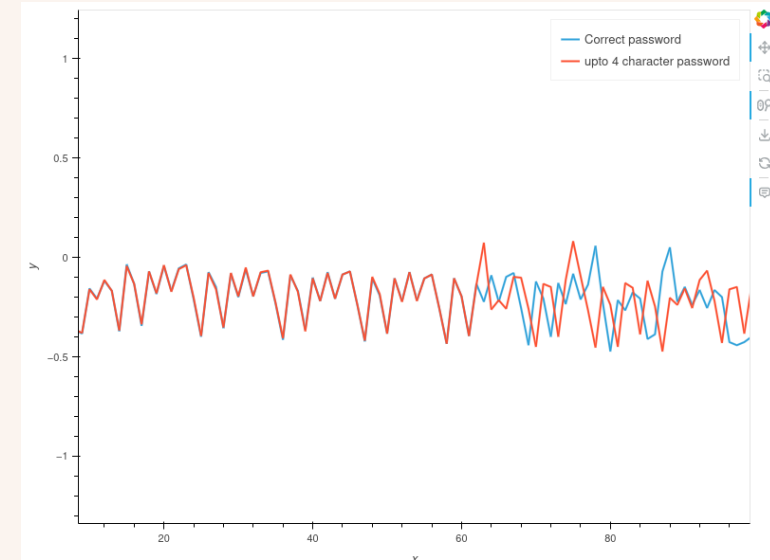
Two separate captures of different instructions:

- **Addition**
- **Multiplication**
- Multiplication consumes more clock cycles than addition.
- The extra cycles cause a longer region in the power trace.
- ChipWhisperer can measure these differences very clearly.
- Instruction behavior directly leaks into power consumption.

POWER ANALYSIS FOR PASSWORD BYPASS

The main idea:

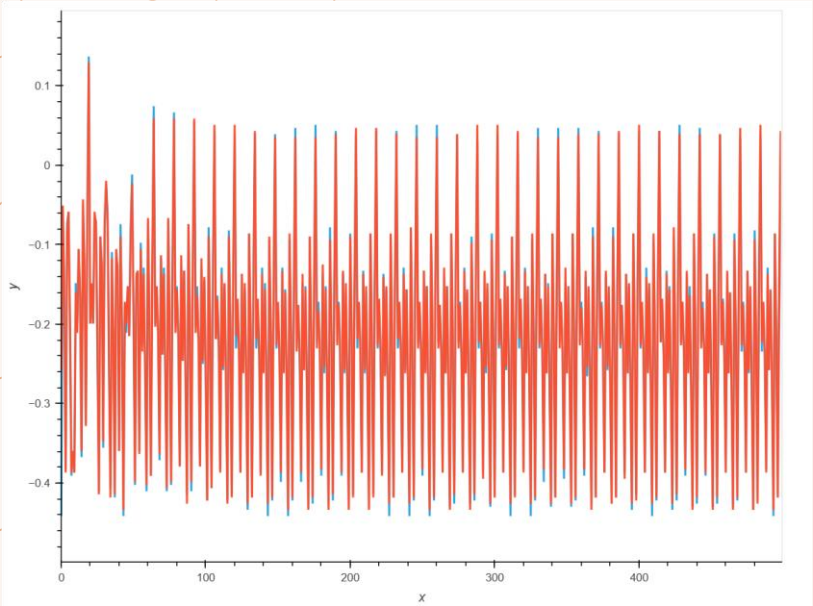
Correct and incorrect characters take different code paths → different power patterns.



To guess a single password character, the idea is simple:

1. Capture a “reference” trace using a known wrong character
2. Try all possible characters
3. Compare each character's trace with the reference
4. The correct character will show the **largest difference**

POWER ANALYSIS FOR PASSWORD BYPASS

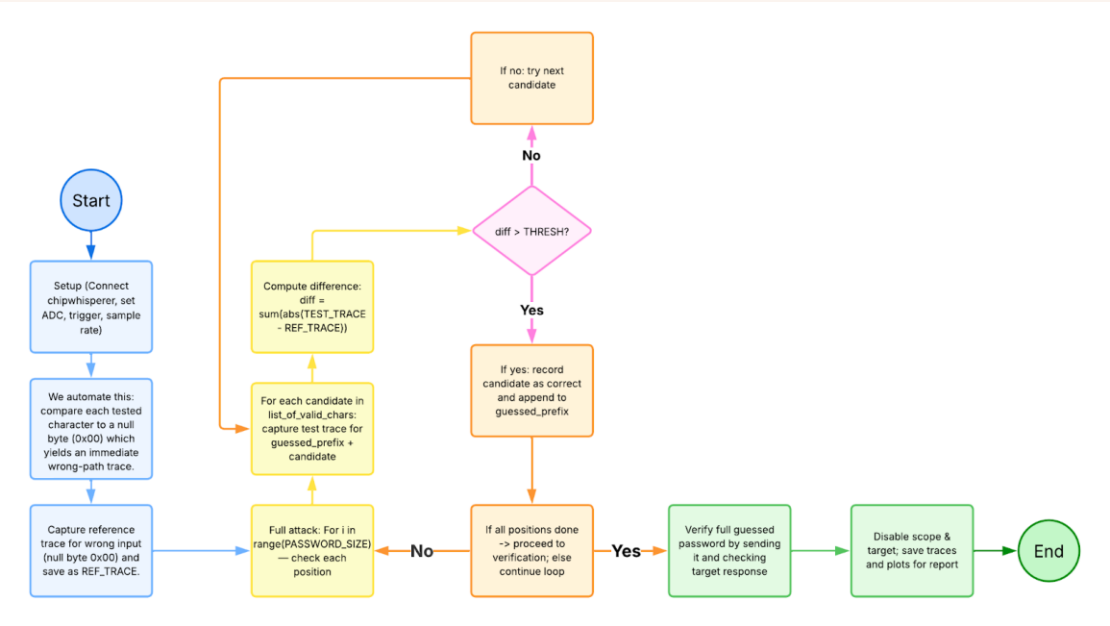


A wrong character produces almost the same trace as the null-byte reference.

complete process for password recovery

Threshold	% Characters Selected	Meaning
5	90%	Threshold too low — selects almost everyone
15	10%	Still too low, many false candidates
25	2%	Good region — only correct character stands out
30	0%	Threshold too high — selects none
40	0%	Definitely too high

Threshold vs % of characters selected



LARGE HAMMING WEIGHT SWINGS

To show that **not only instructions**, but also the **data** being processed by the microcontroller affects the power trace.

Hamming weight = number of bits that are 1 in a byte.

0x00 → **00000000** → **HW = 0**

0xFF → **11111111** → **HW = 8**

A value like 0xFF has **many bits switching**, which usually means **more power consumption** during internal operations.

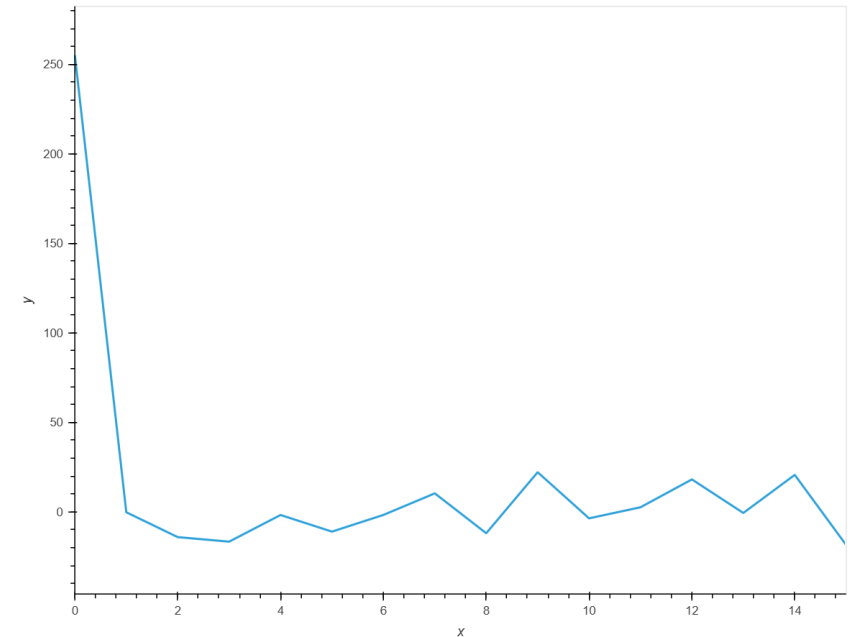
This is why the experiment sends two extreme cases:

All zeros → **low switching** → **low power**

All ones → **high switching** → **higher power**

This proves that **power consumption leaks information about the data being processed**, not just the instructions.

Capturing Traces for Two Extreme Inputs, averaging and comparing the averages:



many bits flips at start → more switching → higher power

RECOVERING DATA FROM A SINGLE BIT

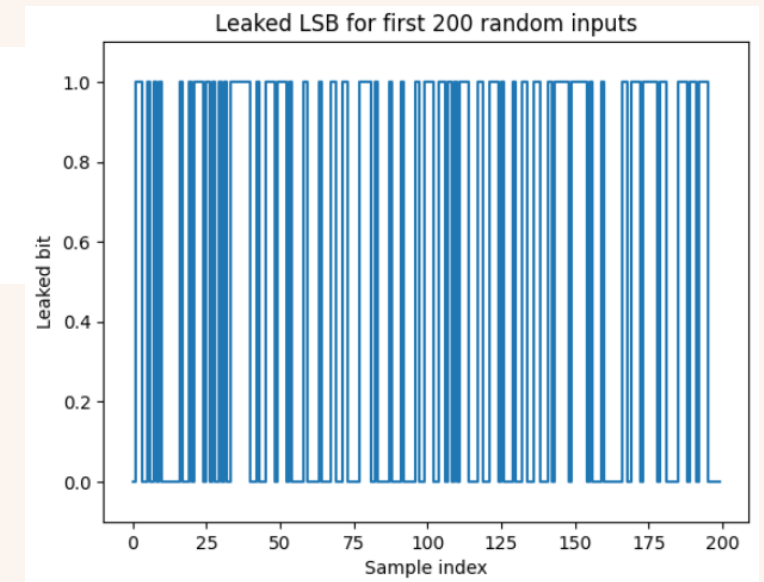
Even if a device leaks only 1 bit of information from the AES S-Box output, that tiny leak is enough to recover the secret key.

$SBox[plaintext_byte \oplus key]$



1. Choose a secret key (fixed to $0 \times EF$ here).
2. Generate many input plaintext bytes (0–255).
3. Compute: $SBox[input \oplus key] \rightarrow \text{take } LSB$
4. This leaked 0/1 is what “device” gives us.
5. For each possible key guess (0–255), predict what the leakage **should** be.
6. The correct key guess will match the real leakage the **highest number of times**.

This is exactly how correlation/DPA works in real SCA.



```
Key Guess EF = 1000 matches
Key Guess 89 = 0583 matches
Key Guess A6 = 0577 matches
Key Guess F9 = 0569 matches
Key Guess D7 = 0569 matches
```

DIFFERENTIAL POWER ANALYSIS (DPA) ON FIRMWARE AES

DPA is a side-channel attack technique where you recover a secret key by comparing small power differences while a device runs encryption such as AES.

Break the AES key using **actual power traces** captured from the ChipWhisperer Nano.



How DPA Works

DPA uses this formula:

intermediate value =

$\text{SBox}[\text{plaintext_byte} \oplus \text{key_byte_guess}]$

Then:

Extract one bit (usually **LSB**)

Separate traces based on this predicted bit

→ Group 1 (predicted bit = 1)

→ Group 0 (predicted bit = 0)

Average both groups

Take absolute difference

Correct guess → strong peak where S-Box occurs

Wrong guess → no peak

DIFFERENTIAL POWER ANALYSIS (DPA) ON FIRMWARE AES

We capture:

- 2500 traces for more accurate grouping
- Each trace ~5000 data points
- Each trace corresponds to one AES encryption

With DPA we capture predicted traces for each key_guess(0-255)

`predicted=SBox[plaintext_byte $\oplus$ key_guess]`

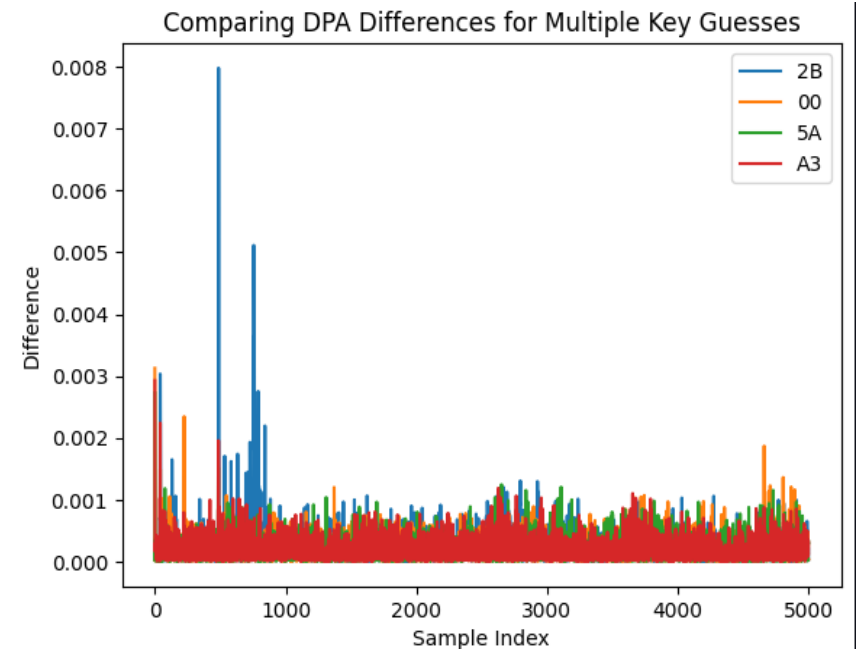
Extract LSB:

- If LSB=1 → put trace in **Group 1**
- If LSB=0 → put trace in **Group 0**

`zero_avg = mean(Group0)`

`one_avg = mean(Group1)`

`diff = abs(one_avg - zero_avg)`



Correct key guess → highest peak
Wrong guesses → smaller values

DIFFERENTIAL POWER ANALYSIS (DPA) ON FIRMWARE AES

Recovering All 16 AES Key Bytes

The attack is repeated for

byte 0 → find correct key

byte 1 → find correct key

...

byte 15 → find correct key

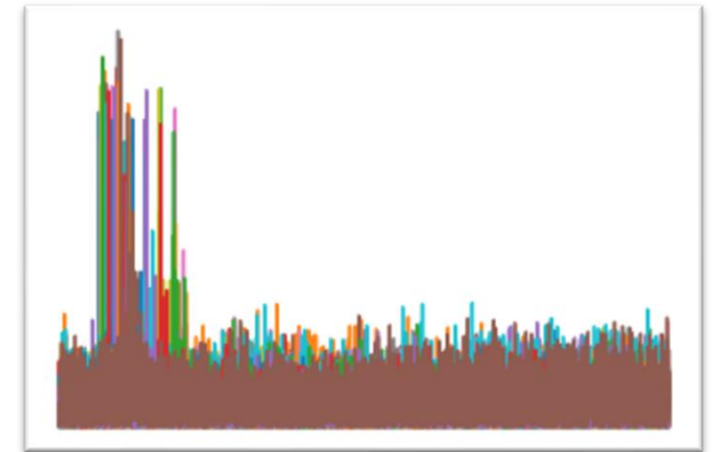
Expected output:

True

2b 7e 15 16 28 ae d2 a6 ab f7 15 88 9 cf 4f 3c

Ghost Peaks

wrong guesses also produce high peaks



Solutions:

1. Increase the no. of traces
2. Try different bits
3. Window the data

KEY FINDINGS

- Power leakage is significant and observable
- Instructions and branches leak instantly
- Hamming Weight strongly influences power
- DPA recovered the full AES key successfully
- Trace preprocessing helps expose hidden leakage

LIMITATIONS

- ChipWhisperer-Nano has lower sampling resolution than the Pro model
- Experiments were done on unprotected firmware (no masking/hiding)
- Trace misalignment and noise affect results
- Larger or more complex algorithms can be loaded on the target, but:
 - they run slower
 - require more memory
 - produce longer traces
 - and usually need many more captures to attack
- Real secure devices use countermeasures that make attacks harder

FUTURE WORK

- **Correlation Power Analysis (CPA):** Use statistical correlation instead of difference-of-means to recover keys more reliably.
- **Exploring stronger leakage models:** Hamming Distance, multi-bit leakage, or S-Box output-based models.
- **Higher-order attacks:** Useful when devices use basic masking countermeasures.
- **Fault Injection (Glitching):** Injecting clock/voltage faults to skip instructions or break security checks.
- **Machine-Learning based SCA:** Using neural networks to learn leakage patterns directly from traces.
- **Testing secure or optimized firmware:** Flashing bigger algorithms or countermeasure-enabled AES to see how leakage changes.

A series of thin, light-brown lines forming an abstract geometric pattern in the top-left corner of the slide. The lines intersect to create various triangular and polygonal shapes.

THANK YOU

All labs and experiments are based on the ChipWhisperer educational content by **NewAE Technology Inc.**

Thanks **to Prof. Sharad Sinha** for guidance.

Thanks to **IIT Goa** for providing resources.